Improving the Clause-Density Dependence of Exact k-SAT Counting

Research note prepared for Vladimir Reshetnikov

ChatGPT research assistance

October 26, 2026

Abstract

We propose a modification of the exact counting algorithm in OpenAI's *The Exponential Time Hypothesis is False*, revised October 25, 2026. Using its Boolean tensor-contraction theorem and parity-lift identity, and retaining its general counter at high density, we derive, for fixed $k \geq 2$, a deterministic polynomial-space algorithm whose bit complexity on $n \geq 20$ declared variables and m clauses is

$$(n+m)^{O_k(1)} 2^{O_k\left(\frac{n}{\sqrt{\log n}} \min\{\log(2+m/n), \log\log n\}\right)}.$$

The change is to construct the initial sparse parity checks using a powered constant-degree expander with local-code parameters depending on clause density, and to keep the actual monomial sparsity in the source's incidence-encoding bound. For $m = \Theta(n \log\log n)$ this replaces an exponent $O_k(n \log\log n/\sqrt{\log n})$ by $O_k(n \log\log\log n/\sqrt{\log n})$. This is an improvement of the density dependence. The unrestricted worst-case bound in n, and the fixed-density exponent, remain those of the source. We give explicit integer parameters, correctness and complexity proofs, a boundary analysis, and reproducible finite checks. The extension is a mathematical derivation relative to the cited source theorems; it has not been formally verified in Lean or Rocq.

Contents

1 Result, comparison, and provenance **2**

2 A sparse-polynomial counter that retains its sparsity **3**

3 Initial parity checks with support depending only on density **5**

4 From sparse checks to exact counting **8**

5 The full algorithm and its uniform complexity **9**

6 Where the bound improves, and where it does not **10**

7 Verification status and reproducible artifacts **12**

8 Further research questions **12**

1 Result, comparison, and provenance

All logarithms in this note are base two unless explicitly written ln. Changing the base affects only fixed constants. The parameter k is fixed; the density may grow with n. The count includes every declared variable, whether or not it occurs in a clause.

The revised source [1] states the general bound

$$(n+m)^{O_k(1)}2^{O_k(n\log\log n/\sqrt{\log n})} \tag{1.1}$$

and, for each fixed D, the bound

$$(n+m)^{O_{k,D}(1)}2^{O_{k,D}(n/\sqrt{\log n})} \qquad (m \le Dn). \tag{1.2}$$

The stronger tensor-contraction appendix, rather than the earlier rational-interpolation construction, is the baseline here. In fact its direct clause-network proof also gives $2^{O_k((n+m)/\sqrt{\log(n+m)})}$ up to a polynomial input factor. Combining this with (1.1), for $B = \max\{1, \lceil m/n \rceil\} \le \lceil \log n \rceil$, gives the exponent scale

$$\frac{n}{\sqrt{\log n}} \min\{1+B, \log\log n\}. \tag{1.3}$$

Keeping the density information through the parity reduction improves this profile.

Theorem 1.1 (Density-dependent improvement)**.** *Use the Boolean tensor-contraction theorem, parity-lift identity, and general-counting bound of [1] as imported results. For each fixed integer $k \ge 2$, there is a deterministic algorithm that exactly counts the satisfying assignments of a k-CNF on $n \ge 20$ declared variables with m input clauses in*

$$(n+m)^{O_k(1)}2^{O_k\left(\frac{n}{\sqrt{\log n}}\min\{\log(2+m/n),\log\log n\}\right)} \tag{1.4}$$

bit operations and $(n+m)^{O_k(1)}$ bits of space. All constants are independent of m/n.

The proof constructs a new reduction for $B \le \lceil \log n \rceil$ and uses the source's general algorithm for larger B. A more general version works for $B \le (\log n)^A$ for any fixed A, with constants allowed to depend on A. Only $A = 1$ is needed for Theorem 1.1.

Corollary 1.2. *For $m = \Theta(n \log\log n)$, the improved bound is*

$$(n+m)^{O_k(1)}2^{O_k(n\log\log\log n/\sqrt{\log n})}.$$

More generally, if $B \to \infty$ and $\log B = o(\log\log n)$, the new exponent scale is little-oh of (1.3).

Proof. Substitute the indicated density into (1.4). For the second claim, both $\log(B+2)/(1+B)$ and $\log(B+2)/\log\log n$ tend to zero. The ratio to the minimum in (1.3) is the maximum of these two ratios. □

What is inherited and what changes. The contraction algorithm and its $s/\sqrt{\log s}$ exponent are inherited. The parity-lift identity and the monomial-to-network translation are also inherited. The new ingredients in this comparison are the density-only bounds on check support, and the uniform retention of the actual sparsity in the network-size estimate. Expander codes themselves are classical and already appear in the source. This note does not claim a new coding paradigm or priority over unexamined work.

Version examined. The source snapshot is `openai/math` at commit

`7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`.

The analysis uses three sources in the preprint's `build/` directory:

- `main_full_version.tex`;
- `quantitative-reduction.tex`;
- `quantitative-contraction.tex`.

The accompanying source manifest records their Git blob hashes and permanent URLs. This note does not report rerunning the source's formal verification or implementing its complete contraction engine.

2 A sparse-polynomial counter that retains its sparsity

The first step is a quantitative refinement of the source's lemma headed "Counting sparse polynomial systems by contraction." Its proof gives more information than its stated polylogarithmic-sparsity bound when the actual number of monomials is retained.

The imported contraction interface allows a bipartite Boolean tensor network with $O(s)$ edges and maximum degree at most $\left\lceil s^{9/10} \right\rceil$, over $(\mathbb{Z}/2^p\mathbb{Z})[V]/(V^\ell)$ with $p, \ell = O(s)$. There must be polynomially many vertices and a description of length $\mathcal{L}$. If each tensor entry is computable in $(s+\mathcal{L})^{O(1)}2^{O(s^{9/10})}$ time and $(s+\mathcal{L})^{O(1)}$ space, the source theorem contracts the network in $(s+\mathcal{L})^{O(1)}2^{O(s/\sqrt{\log s})}$ time and polynomial space. All constants in this interface are fixed. This is the input whose hypotheses are checked below.

Lemma 2.1 (Uniform sparsity dependence)**.** *Fix positive integers k, c, C. Let $P_1, \dots, P_a$ be polynomials over $\mathbb{F}_2$ on $n \geq 20$ declared Boolean variables, where $a \leq cn$. Each polynomial is given by at most S monomials of degree at most k, with*

$$1 \leq S \leq \lambda^C, \qquad \lambda = \lceil \log n \rceil .$$

Under the source assumptions of Theorem 1.1, their number of common zeros can be computed deterministically in polynomial space and

$$n^{O_{k,c,C}(1)}2^{O_{k,c,C}(n\log(S+2)/\sqrt{\log n})} \tag{2.1}$$

bit operations. The constants are uniform over all permitted S.

Proof. Replace positive powers of each Boolean variable by its first power, and cancel duplicate monomials modulo two. There are at most

$$N_0 \leq akS \leq cknS \tag{2.2}$$

equation–variable incidences. The scope of one equation has size at most kS.

Control the large variable degrees. Set

$$D_* = \max\{1, \lceil N_0/n \rceil\} \left\lceil \sqrt{\lambda} \right\rceil .$$

Branch over all assignments to variables belonging to more than D_* scopes. There are at most $n/\sqrt{\lambda}$ such variables, by counting incidences. Thus branching contributes at most $2^{n/\sqrt{\lambda}}$ calls. Substitute the fixed bits, normalize again, and discard constant-zero equations. A constant-one equation gives zero immediately. All remaining variable incidences are bounded by $O_{k,c}(S\sqrt{\lambda})$; every remaining equation still has at most S monomials. Keep the original value of n for all bounds and precisions.

Use the source's bounded-degree lift. The source's parity-lift construction works over

$$\mathcal{B} = (\mathbb{Z}/2^{n+1}\mathbb{Z})[V]/(V^{n+1}).$$

Give each surviving incidence (i, v) its own variable z_{iv}. The construction produces a multilinear polynomial

$$F(z, V) = \prod_i \overline{B}_i(z_i, V)$$

with two properties. First, every retained incidence monomial has degree at most

$$K = k(a + 2n), \qquad 2kn \le K \le (c + 2)kn. \tag{2.3}$$

Second, after identifying every z_{iv} with the original Boolean x_v, summing over the free Boolean assignments, and summing the V-coefficients, the result is congruent modulo 2^{n+1} to the branch count. These are precisely the degree and residue identities in the source's sparse-query proof. They use its total-order parity lift; no new parity approximation is being assumed here.

Encode each monomial by subsets of incidences. Partition the tests and the free variables separately into $J = \left\lceil n^{1/4} \right\rceil$ groups, balanced in cardinality. Let N_{gh} be the number of incidences between test group g and variable group h, and let $N = \sum_{g,h} N_{gh} \le N_0$. Enumerate arrays (d_{gh}) with $0 \le d_{gh} \le N_{gh}$ and $\sum d_{gh} \le K$. There are at most

$$(K + 1)^{J^2} = 2^{O_{k,c}(\sqrt{n} \log n)}$$

such arrays. For a fixed array, use $\left\lceil \log \binom{N_{gh}}{d_{gh}} \right\rceil$ Boolean wires to encode the chosen incidence subset for each pair of groups. Invalid indices receive tensor value zero. The source's tensor entries multiply the indicated test coefficients and sum the identified variable powers; therefore summing these networks over the degree arrays computes the required lift sum exactly.

The important estimate is the number of wires M:

$$\begin{aligned} M &\le J^2 + \log \prod_{g,h} \binom{N_{gh}}{d_{gh}} \\ &\le J^2 + \log \binom{N + K}{K} \\ &\le J^2 + K \log\left(\frac{\mathrm{e}(N + K)}{K}\right) = O_{k,c}(n \log(S + 2)). \end{aligned} \tag{2.4}$$

The second line bounds the number of the specified squarefree monomials by the number of all monomials of degree at most K in N variables. The last line uses (2.2) and (2.3). Replacing S by an unspecified power of $\log n$ at this point would discard the improvement.

Check the contraction hypotheses uniformly. Choose the integer size parameter

$$\tau = 1 + \lceil \log(S + 1) \rceil, \qquad s = n\tau.$$

Then (2.4) gives $M \le C_0 s$ for a fixed $C_0 = C_0(k, c)$. There are $2J$ vertices. Their degrees are at most

$$O_{k,c}(n^{3/4} S \sqrt{\lambda}) = o(s^{9/10}),$$

uniformly for $S \le \lambda^C$. Each test-group tensor entry is computed by subset Möbius inversion, one test at a time, in $2^{kS} n^{O_{k,c,C}(1)}$ time and polynomial space. This is at most $n^{O_{k,c,C}(1)} 2^{s^{9/10}}$ for all sufficiently large n, uniformly over the same range. Variable-group entries and index decoding have polynomial cost. The graph and the entry descriptions have polynomial length; the precision and truncation order are $n + 1 = O(s)$.

The source tensor theorem thus applies with fixed constants. It gives time polynomial in the description length times

$$2^{O_{k,c,C}(s/\sqrt{\log s})} = 2^{O_{k,c,C}(n \log(S+2)/\sqrt{\log n})}.$$

Both the high-incidence branching exponent $n/\sqrt{\lambda}$ and the array-enumeration exponent $O(\sqrt{n} \log n)$ are absorbed in this bound. Process branches, arrays, and contractions sequentially. The source's depth-first polynomial-space argument applies unchanged. The final count lies in $[0, 2^n]$, so its least nonnegative residue modulo 2^{n+1} is the exact integer count. A finite range of smaller n, depending only on k, c, C, can be handled by direct enumeration. This changes only fixed constants. □

3 Initial parity checks with support depending only on density

The source's initial checks use, with $m \le Bn$ and $\mu = \lfloor \log m \rfloor + 1$,

$$D_0 = 320000 B^2 \mu^3, \qquad P_0 = 20000 B^2 \mu^2.$$

Here D_0 bounds initial row supports and P_0 is the later sampling length. These factors of μ make the query sparsity a power of $\log n$ even when B grows very slowly. We replace this part of the construction.

3.1 The explicit expander input

For every integer $q \ge 1$, the Margulis–Gabber–Galil graph on $(\mathbb{Z}/q\mathbb{Z})^2$ has the following eight labelled ports from (x, y):

$$(x \pm 2y, y), \quad (x \pm (2y+1), y), \quad (x, y \pm 2x), \quad (x, y \pm (2x+1)). \tag{3.1}$$

All operations are modulo q. Parallel edges and self-loops retain their port identities. The classical spectral theorem bounds the second largest eigenvalue of its normalized adjacency matrix $A/8$ by $5\sqrt{2}/8$ [2, 3]. The cited formalization also records the explicit maps and the spectral proof.

Add eight labelled identity ports and use the degree-16 lazy operator

$$Q = \frac{I + A/8}{2}.$$

Every eigenvalue is nonnegative, and its restriction to the orthogonal complement of the constants has norm at most

$$\rho = \frac{1 + 5\sqrt{2}/8}{2} < \frac{19}{20}. \tag{3.2}$$

This laziness avoids any convention about whether "second eigenvalue" bounds an absolute value. In the t-step walk graph, the degree is 16^t and the corresponding norm is at most $(19/20)^t$. A port is a word of length t in the 16-symbol alphabet. Each step map is a permutation, so there are exactly 16^t incoming and outgoing walk ports at every vertex.

3.2 Integer parameters and the small-instance branch

Let $B \geq 1$ be an integer with $m \leq Bn$. Define

$$b = \lceil \log(2B) \rceil, \qquad A_B = 4096Bb, \qquad \delta = A_B^{-1}, \qquad P = 2A_B^2. \tag{3.3}$$

Choose the least integer $t \geq 1$ satisfying

$$2A_B 19^t \leq 20^t, \qquad 16^t \geq 64B, \qquad \text{and set } d = 16^t. \tag{3.4}$$

No real-number oracle is needed. These integer inequalities imply $\varepsilon = (19/20)^t \leq \delta/2$ and $d \geq 64B$. They also give

$$t = O(\log(B+2)), \qquad d \leq (16384Bb)^{56}. \tag{3.5}$$

Indeed $2 \cdot 19^{14} < 20^{14}$, so $t = 14\,\lceil \log(8192Bb) \rceil$ is an admissible, possibly nonminimal choice. Since $2^{\lceil \log x \rceil} < 2x$ for $x > 0$ except at harmless equality endpoints, its degree is bounded as in (3.5). This same choice also satisfies $16^t \geq 64B$.

If $m \leq d$, directly enumerate the assignments of the at most km occurring variables and multiply the resulting count by $2^{n-n_{\mathrm{occ}}}$. This costs

$$\mathrm{poly}_k(n,m)2^{kd} = \mathrm{poly}_k(n,m)2^{(B+2)^{O_k(1)}}. \tag{3.6}$$

When $B \leq \lceil \log n \rceil$ this is an asymptotically subordinate overhead. It also avoids padding a tiny formula to an enormous graph. The constants in (3.5) are deliberately conservative; they are not intended to provide a competitive practical solver.

Assume for the rest of this section that $m > d$. Set

$$q = \left\lceil \sqrt{m/d} \right\rceil, \qquad N = q^2, \qquad M = Nd. \tag{3.7}$$

The integer q can be found as the least positive integer satisfying $q^2 d \geq m$. Since $m/d > 1$, we have

$$m \leq M < 4m. \tag{3.8}$$

Take a left and a right copy of the N vertices of the powered lazy graph. Each labelled walk gives an edge from its left starting vertex to its right ending vertex. This is a d-regular bipartite multigraph. Spectral decomposition gives, for left and right vertex sets U, V,

$$e(U,V) \leq \frac{d|U||V|}{N} + \varepsilon d\sqrt{|U||V|}. \tag{3.9}$$

To obtain it, subtract the constant components of the two indicator vectors and bound the residual bilinear form by its operator norm. Replacing the residual norms by $\sqrt{|U|}$ and $\sqrt{|V|}$ only weakens the estimate.

3.3 A local code with density-dependent distance

Lemma 3.1. *There exists a binary matrix H with d columns and $c_{\mathrm{loc}} = \lceil d/(64B) \rceil$ rows such that every nonzero word in its kernel has Hamming weight at least δd. Such a matrix can be found deterministically in $2^{O(d^2)}$ bit operations and $O(d^2)$ working bits.*

Proof. For $0 < x \le 1/2$, let $H_2(x) = -x\log x - (1-x)\log(1-x)$ be binary entropy. The usual binomial bound and $H_2(x) \le x\log(\mathrm{e}/x)$ give

$$\sum_{j=0}^{\lceil \delta d\rceil - 1} \binom{d}{j} \le 2^{dH_2(\delta)}.$$

Using $\log B \le b-1$, $\log b \le b$, $b \ge 1$, and $\log \mathrm{e} < 2$, we have

$$\log(\mathrm{e}/\delta) = \log \mathrm{e} + 12 + \log B + \log b < 13 + 2b \le 16b.$$

Consequently

$$H_2(\delta) \le \frac{1}{256B} < \frac{1}{64B} \le \frac{c_{\mathrm{loc}}}{d}. \tag{3.10}$$

For a uniformly random c_{loc}-by-d binary matrix, every fixed nonzero vector belongs to its kernel with probability $2^{-c_{\mathrm{loc}}}$. The expected number of nonzero kernel words of weight below δd is strictly less than one by (3.10). Some matrix has no such word. Enumerate the matrices lexicographically, testing the forbidden words for each. The total cost is at most $2^{c_{\mathrm{loc}}d+d}\,\mathrm{poly}(d)$, which is $2^{O(d^2)}$, and only the current matrix and word are needed in working storage. The test $w < \delta d$ is the exact integer test $A_B w < d$. □

3.4 Global checks, distance, and shortening

Place the local checks H on the incident ports of every vertex on both sides of the bipartite graph. The resulting global matrix has

$$\begin{aligned} 2Nc_{\mathrm{loc}} &\le \frac{M}{32B} + \frac{2M}{d} \\ &\le \frac{M}{16B} < \frac{m}{4B} \le \frac{n}{4} \end{aligned} \tag{3.11}$$

rows. Each row reads at most d edge bits. The padding factor in (3.8) is explicitly paid for in this estimate.

Lemma 3.2. *When $m > d$, the construction gives a deterministic binary matrix $A^{(0)}$ with m columns and fewer than $n/4$ rows, each of support at most d, satisfying*

$$0 \ne z \in \ker_{\mathbb{F}_2} A^{(0)} \quad \Longrightarrow \quad |\operatorname{supp} z| \ge m/P. \tag{3.12}$$

Here d, P are bounded by fixed polynomials in $B+2$.

Proof. First consider a nonzero accepted word on all M edges. Let w be its weight and U, V the vertex sets touched by its nonzero edges. The local-code property gives

$$w \ge \delta d \max\{|U|, |V|\} \ge \delta d\sqrt{|U||V|}.$$

Every nonzero edge joins U to V, so $w \le e(U,V)$. Combining with (3.9) and dividing by the positive number $d\sqrt{|U||V|}$ yields

$$\sqrt{|U||V|} \ge (\delta - \varepsilon)N.$$

It follows that

$$w \ge \delta(\delta-\varepsilon)Nd \ge \frac{\delta^2 M}{2} \ge \frac{m}{P}.$$

Keep any m labelled edge coordinates and fix the remaining ones to zero. Restrict every global row to the retained columns to obtain $A^{(0)}$. A word in the restricted kernel extends by zeros to a word in the original kernel. Its weight is unchanged, so the distance lower bound survives this shortening. Row count and support size cannot increase. Equations (3.3) and (3.5) prove the claimed polynomial dependence on B. □

Constructing the graph and its port lists uses polynomial time in n, m, d, t. Searching for the local matrix costs $2^{O(d^2)}$ once. The graph has fewer than $4m$ edges. The whole construction uses space polynomial in n, m, d, hence polynomial in n, m in the density range used below.

4 From sparse checks to exact counting

For a clause e, let $v_e(x) \in \mathbb{F}_2$ indicate its violation by assignment x. As a Boolean polynomial it has degree at most k and at most 2^k monomials. Let $v(x) = (v_1(x), \dots, v_m(x))$. Given a binary check matrix A, define

$$N(A) = \big|\{x \in \{0,1\}^n : Av(x) = 0\}\big|.$$

Every satisfying assignment survives every linear check. Therefore

$$N(A) = \#F + D(A), \tag{4.1}$$

where $D(A)$ counts the surviving nonsatisfying assignments.

Lemma 4.1 (Greedy elimination)**.** *Starting with $A^{(0)}$ from Lemma 3.2, a deterministic oracle reduction appends $3n$ rows of list length P and obtains $N(A) = \#F$. It makes at most $3n(2m)^P + 1$ queries to an exact common-zero counting oracle. Every query has fewer than $4n$ equations, of degree at most k, and at most*

$$S_B = 2^k \max\{d, P\} \tag{4.2}$$

monomials in each equation.

Proof. Represent a candidate row by an ordered list

$$\gamma = ((e_1, c_1), \dots, (e_P, c_P)) \in ([m] \times \{0,1\})^P.$$

Repeated clause indices are allowed; their coefficients are added modulo two. Fix a nonsatisfying survivor x of the current matrix. Its violation vector remains in the kernel of $A^{(0)}$ and has support at least m/P. A uniform candidate list misses this support with probability at most

$$(1 - 1/P)^P \le e^{-1} < 1/2.$$

Conditional on a hit, an independent coefficient at a hit position makes the parity uniform, even if indices repeat. Thus at least one quarter of the candidate lists reject x. Summing over all nonsatisfying survivors gives

$$\mathbb{E}_\gamma D(A;\gamma) \le \tfrac{3}{4} D(A).$$

Enumerate every candidate and compute $N(A;\gamma)$ using the exact common-zero counting oracle. Append a minimizing row. Equation (4.1) shows that this also minimizes D even though $\#F$ is not known. After $3n$ rounds,

$$D(A) \le (3/4)^{3n} 2^n = (27/32)^n < 1.$$

Since $D(A)$ is a nonnegative integer, it is zero. One final query returns $\#F$.

The initial row count is below $n/4$; at most $3n$ rows are ever appended. Every tested system therefore has fewer than $4n$ equations. A row supported on at most d clauses expands to at most $2^k d$ monomials, and a list of length P expands to at most $2^k P$ monomials. Cancellation only decreases these numbers. This proves (4.2) and the query bound. □

The argument is the source's greedy selection step with new support parameters. Its use of exact common-zero counts is not circular: Lemma 2.1 depends on the already available tensor counter, not on the density-dependent SAT algorithm being constructed.

5 The full algorithm and its uniform complexity

Proof of Theorem 1.1. Normalize the input by deleting tautologies and repeated clauses and literals, without changing the list of declared variables. An empty clause gives zero; a formula with no clauses gives 2^n. Let m' be the number of remaining clauses and $B = \max\{1, \lceil m'/n \rceil\}$. Since $m' \le m$, bounds expressed in terms of m' are no larger, up to the input-processing polynomial, than the claimed bounds in terms of m.

Suppose first that $B \le \lambda = \lceil \log n \rceil$. Use (3.3)–(3.4). If $m' \le d$, use the direct occurring-variable branch (3.6). Otherwise construct the checks and use Lemma 4.1. By (3.5),

$$S_B = (B+2)^{O_k(1)}, \qquad \log(S_B + 2) = O_k(\log(B+2)). \tag{5.1}$$

For one fixed exponent $C = C(k)$, all these sparsities are at most λ^C for all sufficiently large n. Lemma 2.1 with $c = 4$ therefore gives query exponent

$$O_k\left(\frac{n \log(B+2)}{\sqrt{\log n}}\right), \tag{5.2}$$

with constants independent of the actual value of B.

The logarithm of the number of queries, the local-code search cost, and the small-instance enumeration cost are bounded by

$$O_k(P \log(2m') + d^2 + kd + \log n) = \lambda^{O_k(1)}. \tag{5.3}$$

For every fixed C', $\lambda^{C'} = o(n/\sqrt{\lambda})$. Hence (5.3) is absorbed in (5.2), uniformly over $B \le \lambda$. Multiplication by the number of queries adds their logarithm to the exponent; it does not multiply the exponential scales. The time is therefore

$$(n+m)^{O_k(1)} 2^{O_k(n \log(B+2)/\sqrt{\log n})}. \tag{5.4}$$

If $B > \lambda$, use the source algorithm (1.1). In this range $\log(2 + m'/n) = \Omega(\log \log n)$, so its exponent agrees, up to a fixed factor, with the minimum in (1.4). In the first range $\log(B+2) = \Theta(\log(2 + m'/n))$ and it is at most a fixed multiple of $\log \log n$. Thus (5.4) also has the form (1.4). These comparisons prove the time bound.

For space, enumerate matrices and candidates one at a time, keep only the current and best candidate counts, and use the polynomial-space sparse-system counter. The selected matrix has $O(n)$ rows; each list or support has polylogarithmic length in the first range. The graph has fewer than $4m'$ edges; local-code enumeration stores $O(d^2)$ bits. Accumulators use $n+1$ bits for counts. The direct occurring-variable branch is depth first, and the source's general branch already has polynomial space. All claimed space bounds follow. A uniformly finite exceptional range of n is handled directly and absorbed in the fixed constants. □

The same low-density construction works, for any fixed $A > 0$, on $B \le \lambda^A$ with constants depending on k, A. Every support, local search, and candidate-enumeration overhead is still bounded by a fixed power of λ in the exponent. This proves the fixed-A extension mentioned after Theorem 1.1.

5.1 Algorithm in operational form

1. Normalize the CNF, preserving its n declared variables. Handle an empty clause or no remaining clauses immediately.
2. Compute $B = \max\{1, \lceil m'/n \rceil\}$ and $\lambda = \lceil \log n \rceil$. If $B > \lambda$, call the source's general counter and return its result.
3. Compute the exact integer parameters in (3.3)–(3.4). If $m' \le d$, count by enumerating the occurring variables, include the unused-variable factor, and return.
4. Construct the powered lazy expander, search for its local check matrix, impose local checks at both ends, and shorten the edge code to m' coordinates.
5. For $3n$ rounds, enumerate lists in $([m'] \times \{0,1\})^P$. Expand each candidate into degree-k equations and count their common zeros with Lemma 2.1. Append a candidate giving the minimum count.
6. Count the common zeros of the final system and return that integer. Lemma 4.1 proves it equals $\#F$.

No randomized choice remains in this algorithm. Random matrices and random lists are used only to prove that exhaustive deterministic searches find objects with the required properties.

6 Where the bound improves, and where it does not

Table 1 compares the exponential scales after factoring out $n/\sqrt{\log n}$. Polynomial factors and fixed constants are suppressed. These are comparisons of proved upper bounds, not runtime measurements or lower bounds on the source.

Table 1: Clause-density dependence of the exponent.

Density B	Source guarantees combined	This modification
$\Theta(1)$	1	1
$\sqrt{\log\log n}$	$\sqrt{\log\log n}$	$\log\log\log n$
$\log\log n$	$\log\log n$	$\log\log\log n$
$(\log\log n)^2$	$\log\log n$	$\log\log\log n$
$(\log n)^\alpha$, fixed $\alpha > 0$	$\log\log n$	$\log\log n$
Unrestricted	$\log\log n$	$\log\log n$

For the central example $B = \Theta(\log\log n)$, the ratio of the old and new exponent scales tends to infinity like

$$\frac{\log\log n}{\log\log\log n}.$$

This is more than a constant-factor change in the exponential upper bound. It is also narrower than improving the worst case over all m: at unrestricted density, (1.4) retains the source exponent. Nor does it improve the fixed-density $n/\sqrt{\log n}$ scale.

6.1 Why simply retuning sparsification does not go further

The source's disjoint sparsification step, with parameter $0 < \sigma \leq 1$, incurs $2^{\sigma n}$ leaves up to polynomial factors, with an occurrence bound

$$B = O_k([\sigma^{-1} \log(2/\sigma)]^{k-1}).$$

Substituting this bound into (5.4) gives the exponent estimate

$$O_k\left(n\left[\sigma + \frac{\log(2/\sigma)}{\sqrt{\log n}}\right]\right). \tag{6.1}$$

On its permitted polylogarithmic-density range, the natural minimizing scale is $\sigma = \Theta(1/\sqrt{\log n})$. For example, minimizing $\sigma + a\ln(1/\sigma)/\sqrt{\log n}$ for a positive fixed a gives $\sigma = a/\sqrt{\log n}$. Its value has scale $\log\log n/\sqrt{\log n}$. Thus this particular upper-bound calculation still has the source's general scale. This observation is a limitation of the present estimate; it is not an impossibility theorem for other sparsifications or other algorithms. For general fixed k, this substitution uses the fixed-A extension with A chosen sufficiently large in terms of k: the occurrence bound at the indicated σ is polylogarithmic, although it can exceed $\log n$.

6.2 Why a total-degree cutoff alone does not beat the tensor exponent

The source's sharper tensor routine enumerates weighted mode monomials. With M wires, u retained valuation levels, and a packet parameter D, its relevant upper budgets have scales

$$\text{colors per weight } O(M/D), \qquad \text{weight budget } O(MD/u),$$

giving $\exp(O(M/\sqrt{u}))$ tuples. Here $u = \Theta(\log s)$. A tempting suggestion is to additionally impose the ordinary degree bound $|\beta| \leq M$. The following elementary example shows why those two budgets by themselves cannot yield a better asymptotic enumeration bound.

Proposition 6.1 (A counting obstruction, not an algorithmic lower bound)**.** *Let M, D, u be positive integers. Suppose an abstract mode family has $\lfloor M/D \rfloor$ distinct modes of every weight in an interval $[b, 2b]$, where $b = \lfloor D/(4\sqrt{u}) \rfloor$, $M/D \geq 1$, and $D/\sqrt{u}$ is sufficiently large. Then there are $2^{\Omega(M/\sqrt{u})}$ distinct exponent tuples satisfying both $|\beta| \leq M$ and $\sum_j w_j\beta_j < MD/u$.*

Proof. There are $\Theta((M/D)b) = \Theta(M/\sqrt{u})$ modes in this interval. Give each of them an exponent in $\{0, 1\}$ independently and all other modes exponent zero. Even selecting them all uses at most $\lfloor M/D \rfloor (b+1) \leq M$ total degree for sufficiently large parameters. The total weight is at most

$$2b \lfloor M/D \rfloor (b+1) < MD/u$$

for the chosen constant $1/4$, again for sufficiently large parameters. Every subset is therefore allowed, yielding the stated number of tuples. □

This proposition does not assert that all such tuples are nonzero in every actual network. It shows that the two numerical budgets alone cannot justify an improved counting estimate. Additional algebraic structure or a way to sum many modes without enumerating them would be needed.

7 Verification status and reproducible artifacts

The proof was checked independently at the following interfaces: the expander padding and port counts; the strict local-code entropy inequality; the global row budget; the minimum-distance argument under shortening; the greedy elimination invariant; and the uniform contraction hypotheses when the density grows with n. The source's complete tensor proof and its formalization were not reverified in this work.

The archive includes a standard-library-only Python verification script and its machine-readable results. The script performs exact finite checks of the integer parameter inequalities, the explicit graph maps and their inverse ports, small spectral bounds, local and shortened code examples, subset indexing, and greedy parity selection. It also records the scale of the explicit parameters. These checks support implementation details and catch bookkeeping errors; the unbounded assertions are proved in the text or imported from the cited theorems.

The executed check suite passed: 3,248 inverse-port identities; four exact rational spectral certificates for side lengths 2 through 5; five small local-code searches; 156 shortening and zero-extension checks; 2,047 subset rank/unrank cases; 30,000 wire-encoding inequalities; 72 greedy parity rounds; parameter choices for $B = 1, \dots, 1000$; and 27,696 padding and row-budget cases. The spectral certificates use exact rational elimination, not floating-point eigenvalue approximations. These figures describe the finite tests only.

What the code is. It is a reference implementation and verifier for the new reduction's components. It is not a complete implementation or benchmark of the subexponential SAT counter. In particular, it does not replace the source's tensor engine by brute force and then claim the engine's asymptotic running time for that brute-force code.

Reproduction. From the extracted archive directory, run:

```
python3 code/verify_components.py --output verification_results.json
latexmk -pdf -interaction=nonstopmode -halt-on-error density_aware_sat_improvement.tex
```

The manuscript uses only common LaTeX packages. The Python checks require no third-party packages. The source manifest identifies the upstream files used; `PROOF_STATUS.txt` records the proof dependencies and the distinction between finite tests and formal verification. `README.txt` describes each artifact.

Practical limitation. Even at small B, the conservative choice of t can make $d = 16^t$ extremely large, and the local-code search is exponential in d^2. The proof absorbs these costs because $B \le \log n$ and every fixed power of $\log n$ is eventually smaller than $n/\sqrt{\log n}$. That comparison does not predict a usable crossover at realistic input sizes. A practical implementation would need substantially better local constructions and a careful implementation of the source engine.

8 Further research questions

Can the incidence information be compressed further? Our modified checks have $S = (B+2)^{O(1)}$ and therefore cost $O(n \log(B+2))$ wires under the source's subset encoding. A stronger reduction would need to encode or sum the relevant incidence selections using fewer wires, or exploit dependencies among their test coefficients. Merely suppressing the sparsity parameter in the notation would not accomplish this. The sharper bound (2.4), involving the actual total incidence count N, suggests studying nonuniform tests and many repeated scopes.

Can one improve the check-support tradeoff? The explicit degree bound (3.5) is intentionally loose. Better expander families and explicit local codes can reduce its power and avoid exhaustive local search. Such changes could improve constants and feasibility. As long as support remains a fixed power of B, its logarithm remains $\Theta(\log B)$, so this alone would not improve the asymptotic exponent scale proved here.

Can one sum weighted modes collectively? Proposition 6.1 rules out a purely numerical improvement from adding only a total-degree cutoff. It leaves open structured generating-function methods, algebraic compression of packet tensors, or exact summation procedures that avoid treating every allowed mode tuple separately. An improvement here would affect the fixed-density bound as well.

Can density and sparsification be optimized jointly by a new invariant? The objective (6.1) retains the $\log\log n$ loss. To beat the unrestricted source bound through this route, one would need stronger information about the leaves than their maximum occurrence bound, such as a provably smaller incidence entropy aggregated over the entire disjoint tree. A uniform worst-case improvement has not been established here.

Can the extension be mechanized economically? A formal proof could reuse the source's tensor theorem and parity lift as interfaces, then formalize the explicit graph powering, entropy and distance inequalities, shortening, greedy elimination, and the uniform asymptotic comparison. The exact-integer formulation (3.4) removes real arithmetic from parameter selection. This is a proposed formalization path, not a claim of a completed formal proof.

References

[1] OpenAI, *The Exponential Time Hypothesis is False*, October 17, 2026, revised October 25, 2026. Source snapshot: `openai/math`, commit `7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`. See the main theorem; "Initial tests that exclude small violation sets"; "Choosing additional equations by counting common zeros"; and the appendices "A sharper bound for exact counting" and "Proof of the contraction theorem." Permanent repository link.

[2] O. Gabber and Z. Galil, *Explicit constructions of linear-sized superconcentrators*, Journal of Computer and System Sciences **22** (1981), 407–420. doi:10.1016/0022-0000(81)90040-4.

[3] Archive of Formal Proofs, *Expander Graphs*, theory `Expander_Graphs_MGG`, section "Margulis Gabber Galil Construction." The graph definition records the eight affine port maps; `mgg_numerical_radius` establishes the spectral bound. Accessed October 26, 2026. Formal theory and proof.